Django REST Framework (DRF) — Complete Notes

Language: বাংলা + English technical terms

Level: Beginner to Advanced

Goal: Backend Development, Real-world Projects & Interview Preparation

Table of Contents

1. API Guide
2. Requests
3. Responses
4. Views
5. Generic Views
6. ViewSets
7. Routers
8. Parsers
9. Renderers
10. Serializers
11. Serializer Fields
12. Serializer Relations
13. Validators
14. Authentication
15. Permissions
16. Caching
17. Throttling
18. Filtering
19. Pagination
20. Versioning
21. Content Negotiation
22. Metadata
23. Schemas
24. Format Suffixes
25. Returning URLs
26. Exceptions
27. Status Codes

28. Testing
29. Settings
30. Complete CRUD Project
31. Interview Questions

---

1. API Guide

What is Django REST Framework?

Django REST Framework (DRF) হলো Django-এর ওপর তৈরি একটি toolkit, যা REST API তৈরি করতে সাহায্য করে।

Features

- Serialization and deserialization
- Request and response handling
- Authentication and permissions
- Generic views and ViewSets
- Filtering and pagination
- Throttling and caching
- API testing
- OpenAPI schema support

What is REST API?

REST (Representational State Transfer) হলো web API design করার একটি architectural style।

Example endpoints:

Method	Endpoint	Operation
GET	`/api/products/`	List products
POST	`/api/products/`	Create product
GET	`/api/products/1/`	Retrieve product

PUT	`/api/products/1/`	Update product
PATCH	`/api/products/1/`	Partial update
DELETE	`/api/products/1/`	Delete product

Important HTTP methods

- `GET` : Data retrieve
- `POST` : Create resource or submit an action
- `PUT` : Full update or replacement
- `PATCH` : Partial update
- `DELETE` : Delete resource
- `OPTIONS` : Discover supported operations
- `HEAD` : Retrieve headers without response body

---

2. Setup and Installation

Install Django and DRF:

```
pip install django djangorestframework
```

Create project:

```
django-admin startproject config .
python manage.py startapp api
```

Add apps in `settings.py` :

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
```

```
    "rest_framework",
    "api",
]
```

Run migrations:

```
python manage.py migrate
python manage.py runserver
```

---

3. Models

`api/models.py`

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
    stock = models.PositiveIntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Create database tables:

```
python manage.py makemigrations
python manage.py migrate
```

---

4. Requests

DRF-এর `Request` object সাধারণ Django `HttpRequest`-এর ওপর অতিরিক্ত API features দেয়।

Important properties

Property	Purpose

`request.data`	Parsed request body
`request.query_params`	URL query parameters
`request.user`	Current user
`request.auth`	Authentication credentials
`request.method`	HTTP method
`request.FILES`	Uploaded files

Example:

```
@api_view(["GET"])
def product_search(request):
    name = request.query_params.get("name", "")

    products = Product.objects.filter(
        name__icontains=name
    )

    serializer = ProductSerializer(products, many=True)
    return Response(serializer.data)
```

Request:

```
GET /api/products/?name=phone
```

`request.data` vs `request.query_params`

- `request.data` : POST, PUT, PATCH-এর request body
- `request.query_params` : URL-এর query string
- `request.FILES` : Uploaded file data

---

5. Responses

DRF-এর `Response` Python data-কে JSON বা অন্য supported format-এ রূপান্তর করে।

```
from rest_framework.response import Response
from rest_framework import status


def example_response():
    return Response(
        {"message": "Success"},
        status=status.HTTP_200_OK,
    )
```

Create response:

```
return Response(
    {"message": "Product created"},
    status=status.HTTP_201_CREATED,
)
```

Response components

- Response body
- HTTP status code
- Response headers
- Content type

DRF views-এ সাধারণত `Response` ব্যবহার করবে। Plain Django views-এ প্রয়োজন অনুযায়ী `JsonResponse` ব্যবহার করা যায়।

---

6. Views

DRF-এ API views তৈরি করার প্রধান পদ্ধতি:

1. Function-based views
2. APIView
3. Generic views

4. ViewSets

6.1 Function-based view

```
from rest_framework.decorators import api_view
from rest_framework.response import Response


@api_view(["GET"])
def hello_api(request):
    return Response({
        "message": "Hello DRF"
    })
```

`@api_view` function-কে DRF API view-তে রূপান্তর করে।

6.2 APIView

```
from rest_framework.views import APIView
from rest_framework.response import Response


class HelloAPIView(APIView):

    def get(self, request):
        return Response({
            "message": "Hello DRF"
        })

    def post(self, request):
        return Response({
            "received": request.data
        })
```

APIView features

- Authentication
- Permissions
- Throttling
- Exception handling
- Content negotiation

Function-based view vs APIView

Function-based	APIView
`@api_view` ব্যবহার করে	Class-based
Simple endpoints	Complex logic
ছোট API-এর জন্য ভালো	Multiple HTTP methods-এর জন্য সুবিধাজনক

---

7. Generic Views

Generic views common CRUD operations সহজ করে।

Important classes

Class	Purpose
`ListAPIView`	List objects
`RetrieveAPIView`	Retrieve one object
`CreateAPIView`	Create object
`UpdateAPIView`	Update object
`DestroyAPIView`	Delete object

`ListCreateAPIView`	List and create
`RetrieveUpdateAPIView`	Retrieve and update
`RetrieveUpdateDestroyAPIView`	Retrieve, update, delete

Example

`api/serializers.py`

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):

    class Meta:
        model = Product
        fields = [
            "id",
            "name",
            "price",
            "stock",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]
```

`api/views.py`

```
from rest_framework.generics import ListCreateAPIView
from .models import Product
from .serializers import ProductSerializer


class ProductListCreateView(ListCreateAPIView):

```

```
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

This supports:

- `GET /api/products/`
- `POST /api/products/`

Useful methods

`get_queryset()`

```
def get_queryset(self):
    return Product.objects.filter(stock__gt=0)
```

`perform_create()`

```
def perform_create(self, serializer):
    serializer.save()
```

`get_serializer_class()`

Different actions বা conditions অনুযায়ী serializer নির্বাচন করতে ব্যবহার করা যায়।

---

8. ViewSets

ViewSet একটি resource-এর related actions এক class-এ সংগঠিত করে।

ModelViewSet

```
from rest_framework.viewsets import ModelViewSet


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

Default actions

- `list()`
- `retrieve()`
- `create()`
- `update()`
- `partial_update()`
- `destroy()`

Types of ViewSets

Class	Purpose
`ViewSet`	Custom actions
`GenericViewSet`	Generic queryset and serializer features
`ReadOnlyModelViewSet`	List and retrieve
`ModelViewSet`	Full CRUD

Custom action

```
from rest_framework.decorators import action
from rest_framework.response import Response


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    @action(detail=False, methods=["get"])
    def available(self, request):
        products = self.get_queryset().filter(stock__gt=0)

        serializer = self.get_serializer(
            products,
            many=True,
        )

        return Response(serializer.data)
```

Endpoint:

```
GET /api/products/available/
```

`detail=False` collection-level action বোঝায়। `detail=True` হলে action একটি নির্দিষ্ট object-এর ওপর কাজ করে।

---

9. Routers

Router ViewSet-এর জন্য URL patterns স্বয়ংক্রিয়ভাবে তৈরি করে।

`api/urls.py`

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()

router.register(
    "products",
    ProductViewSet,
    basename="product",
)

urlpatterns = router.urls
```

`config/urls.py`

```
from django.urls import path, include

urlpatterns = [
    path("api/", include("api.urls")),
]
```

Generated endpoints

Method	URL	Action
GET	`/api/products/`	List
POST	`/api/products/`	Create
GET	`/api/products/1/`	Retrieve

PUT	`/api/products/1/`	Update
PATCH	`/api/products/1/`	Partial update
DELETE	`/api/products/1/`	Delete

DefaultRouter vs SimpleRouter

- `SimpleRouter` : ViewSet routes তৈরি করে।
- `DefaultRouter` : API root এবং format suffix support-ও দেয়।

---

10. Parsers

Parser incoming request body-কে Python data-তে রূপান্তর করে।

Common parsers

Parser	Use
`JSONParser`	JSON body
`FormParser`	URL-encoded forms
`MultiPartParser`	File uploads
`FileUploadParser`	Raw file upload

Example:

```
from rest_framework.parsers import JSONParser
from rest_framework.views import APIView


class ProductAPIView(APIView):
    parser_classes = [JSONParser]
```

JSON request header:

```
Content-Type: application/json
```

Important distinction

- Parser: Incoming data
- Renderer: Outgoing data

Malformed JSON হলে সাধারণত `400 Bad Request` পাওয়া যায়। Unsupported request content type হলে `415 Unsupported Media Type` হতে পারে।

---

11. Renderers

Renderer response data-কে JSON, HTML বা অন্য supported representation-এ রূপান্তর করে।

Common renderers

- `JSONRenderer`
- `BrowsableAPIRenderer`
- `TemplateHTMLRenderer`
- `StaticHTMLRenderer`

Example:

```
from rest_framework.renderers import JSONRenderer


class ProductAPIView(APIView):
    renderer_classes = [JSONRenderer]
```

JSON response example

```
{
  "id": 1,
  "name": "Keyboard",
  "price": "1200.00",
  "stock": 10
}
```

`BrowsableAPIRenderer` browser-এ interactive API interface দেয়। API-only production deployment-এ এটি প্রয়োজন না হলে বাদ দেওয়া যায়।

---

12. Serializers

Serializer DRF-এর সবচেয়ে গুরুত্বপূর্ণ concepts-এর একটি।

এর দুটি প্রধান কাজ:

1. Python/model data-কে API-friendly representation-এ রূপান্তর করা।
2. Incoming input validate করা।

12.1 ModelSerializer

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):

    class Meta:
        model = Product
        fields = [
            "id",
            "name",
            "price",
            "stock",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]
```

12.2 Serialize one object

```
product = Product.objects.first()

serializer = ProductSerializer(product)

print(serializer.data)
```

12.3 Serialize multiple objects

```
products = Product.objects.all()

serializer = ProductSerializer(
    products,
```

```
    many=True,
)
```

12.4 Validate input

```
serializer = ProductSerializer(data={
    "name": "Mouse",
    "price": "500.00",
    "stock": 20,
})

serializer.is_valid(raise_exception=True)

product = serializer.save()
```

12.5 Update object

```
serializer = ProductSerializer(
    instance=product,
    data={
        "name": "Gaming Mouse",
        "price": "1500.00",
        "stock": 15,
    },
)

serializer.is_valid(raise_exception=True)
serializer.save()
```

12.6 Partial update

```
serializer = ProductSerializer(
    instance=product,
    data={"stock": 25},
    partial=True,
)

serializer.is_valid(raise_exception=True)
serializer.save()
```

12.7 Serializer vs ModelSerializer

Serializer	ModelSerializer
Fields manually define করা যায়	Model থেকে fields generate হয়
Custom data structures -এর জন্য উপযোগী	Django model API-এর জন্য উপযোগী
Create/update logic manually implement করতে হতে পারে	Common model persistence support করে

Important serializer attributes

- `initial_data`
- `validated_data`
- `errors`
- `data`

`data=` দিয়ে serializer তৈরি করলে `is_valid()` চালানোর পর `validated_data` ও `save()` ব্যবহার করবে।

---

13. Serializer Fields

Fields নির্ধারণ করে API কী ধরনের data গ্রহণ বা return করবে।

Common fields

Field	Purpose

`CharField`	Text
`IntegerField`	Integer
`FloatField`	Floating point
`DecimalField`	Decimal numbers
`BooleanField`	True/false
`EmailField`	Email validation
`URLField`	URL validation
`DateField`	Date
`DateTimeField`	Date and time
`ChoiceField`	Fixed choices
`ListField`	List
`DictField`	Dictionary
`JSONField`	JSON-like data
`SerializerMethodField`	Custom read-only output

Field options

```
name = serializers.CharField(
    max_length=100,
    required=True,
)

description = serializers.CharField(
    required=False,
    allow_blank=True,
)

price = serializers.DecimalField(
    max_digits=10,
    decimal_places=2,
    min_value=0,
)
```

Important options

- `required=False` : Field optional
- `allow_null=True` : Accepts `None`
- `allow_blank=True` : Accepts empty string
- `read_only=True` : Output only
- `write_only=True` : Input only
- `default` : Default value

SerializerMethodField

```
class ProductSerializer(serializers.ModelSerializer):
    is_available = serializers.SerializerMethodField()

    class Meta:
        model = Product
        fields = [
            "id",
            "name",
            "stock",
            "is_available",
        ]
```

```
    def get_is_available(self, obj):
        return obj.stock > 0
```

`SerializerMethodField` সাধারণত read-only।

---

14. Serializer Relations

Django model relationships API-তে represent করতে relational fields ব্যবহার করা হয়।

Example models

```
class Category(models.Model):
    name = models.CharField(max_length=100)


class Product(models.Model):
    name = models.CharField(max_length=100)

    category = models.ForeignKey(
        Category,
        on_delete=models.CASCADE,
        related_name="products",
    )
```

14.1 PrimaryKeyRelatedField

```
class ProductSerializer(serializers.ModelSerializer):
    category = serializers.PrimaryKeyRelatedField(
        queryset=Category.objects.all()
    )

    class Meta:
        model = Product
        fields = ["id", "name", "category"]
```

Input:

```
{
  "name": "Keyboard",
  "category": 1
}
```

14.2 Nested serializer

```
class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ["id", "name"]


class ProductSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)

    class Meta:
        model = Product
        fields = ["id", "name", "category"]
```

Nested serializer related object-এর বিস্তারিত output দেয়।

14.3 StringRelatedField

```
category = serializers.StringRelatedField(
    read_only=True
)
```

Related object's `__str__()` representation দেয়।

14.4 SlugRelatedField

```
category = serializers.SlugRelatedField(
    slug_field="slug",
    queryset=Category.objects.all(),
)
```

14.5 HyperlinkedRelatedField

Related object-এর URL representation দিতে ব্যবহার করা হয়।

Important note

Nested serializer-এ `read_only=True` থাকলে input দিয়ে related object assign করা যায় না।

Writable nested relationships-এর জন্য custom `create()` বা `update()` logic লাগতে পারে।

---

15. Validators

Validation invalid input reject করে।

15.1 Field-level validation

```
class ProductSerializer(serializers.ModelSerializer):
```

```
    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError(
                "Price must be greater than zero."
            )
        return value

    class Meta:
        model = Product
        fields = ["id", "name", "price", "stock"]
```

15.2 Object-level validation

```
def validate(self, attrs):
    if attrs.get("stock", 0) < 0:
        raise serializers.ValidationError({
            "stock": "Stock cannot be negative."
        })

    return attrs
```

15.3 Reusable validator

```
def validate_positive(value):
    if value <= 0:
        raise serializers.ValidationError(
            "Value must be positive."
        )
```

Usage:

```
amount = serializers.DecimalField(
    max_digits=10,
    decimal_places=2,
    validators=[validate_positive],
)
```

Built-in validation options

- `min_value`
- `max_value`
- `min_length`
- `max_length`
- `UniqueValidator`

- `UniqueTogetherValidator`

Database-level constraints-ও ব্যবহার করবে, কারণ simultaneous requests-এর race conditions application-level validation দিয়ে সব সময় আটকানো যায় না।

---

16. Authentication

Authentication নির্ধারণ করে request কোন user বা credentials থেকে এসেছে।

Common authentication methods

Method	Description
SessionAuthentication	Django sessions
BasicAuthentication	HTTP Basic credentials
TokenAuthentication	DRF token
JWT	Access and refresh tokens
OAuth2	Authorization framework

16.1 TokenAuthentication

`settings.py`:

```
INSTALLED_APPS += [
    "rest_framework.authtoken",
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
    ],
}
```

Run:

```
python manage.py migrate
```

Client header:

```
Authorization: Token <token>
```

16.2 JWT Authentication

Install:

```
pip install djangorestframework-simplejwt
```

Settings:

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ],
}
```

URLs:

```
from django.urls import path

from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
)

urlpatterns = [
    path(
        "api/token/",
        TokenObtainPairView.as_view(),
```

```
        name="token_obtain_pair",
    ),
    path(
        "api/token/refresh/",
        TokenRefreshView.as_view(),
        name="token_refresh",
    ),
]
```

Client header:

```
Authorization: Bearer <access_token>
```

Security

- HTTPS ব্যবহার করো।
- Tokens নিরাপদে handle করো।
- Access token-এর expiry configure করো।
- Refresh token strategy রাখো।
- Password কখনো plain text-এ store করো না।
- Authentication মানেই সব data access করার permission নয়।

---

17. Permissions

Permissions নির্ধারণ করে user API action করতে পারবে কি না।

Built-in permissions

Permission	Purpose
`AllowAny`	Public access
`IsAuthenticated`	Authenticated users
`IsAdminUser`	Staff users

`IsAuthenticatedOrReadOnly`	Public read, authenticated write
`DjangoModelPermissions`	Model permissions
`DjangoObjectPermissions`	Object-level permission integration

Example:

```
from rest_framework.permissions import IsAuthenticated


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]
```

Custom owner permission

```
from rest_framework.permissions import BasePermission, SAFE_METHODS


class IsOwnerOrReadOnly(BasePermission):

    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True

        return obj.owner_id == request.user.id
```

এই example-এর model-এ `owner` field থাকতে হবে।

Important security rule

Object-level permission list endpoint-এ প্রতিটি record automatically filter করে না।

User-specific queryset:

```
def get_queryset(self):
    return Product.objects.filter(
        owner=self.request.user
    )
```

Authentication vs Permission

- Authentication: Who are you?
- Permission: What are you allowed to do?

---

18. Caching

Caching repeated database queries ও expensive computations কমাতে পারে।

Cache a view

```
from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page
from rest_framework.generics import ListAPIView


@method_decorator(
    cache_page(60 * 5),
    name="dispatch",
)
class ProductListView(ListAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

এটি পাঁচ মিনিট পর্যন্ত response cache করতে পারে।

Manual caching

```
from django.core.cache import cache


def get_products():
    key = "product-list-v1"

    data = cache.get(key)

    if data is None:
```

```
        data = list(
            Product.objects.values(
                "id",
                "name",
                "price",
            )
        )

        cache.set(key, data, timeout=300)

    return data
```

Caching best practices

- Cache invalidation strategy রাখো।
- TTL configure করো।
- Private data shared cache-এ leak হতে দিও না।
- Multi-server deployments-এ shared cache বিবেচনা করো।
- Stale data গ্রহণযোগ্য কি না তা নির্ধারণ করো।

---

19. Throttling

Throttling request rate সীমিত করে।

Settings

```
REST_FRAMEWORK = {
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {
        "anon": "100/hour",
        "user": "1000/hour",
    },
}
```

Common throttle classes

- `AnonRateThrottle`
- `UserRateThrottle`
- `ScopedRateThrottle`

Scoped example:

```
class LoginView(APIView):
    throttle_scope = "login"
```

Settings:

```
REST_FRAMEWORK = {
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.ScopedRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {
        "login": "5/minute",
    },
}
```

Throttling application-level rate control; এটি একা DDoS protection নয়।

---

20. Filtering, Searching and Ordering

Install:

```
pip install django-filter
```

Settings:

```
INSTALLED_APPS += ["django_filters"]
```

Example:

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import filters


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    filter_backends = [
        DjangoFilterBackend,
        filters.SearchFilter,
        filters.OrderingFilter,
    ]

    filterset_fields = ["stock"]
    search_fields = ["name"]
```

```
    ordering_fields = ["name", "price"]
    ordering = ["name"]
```

Example queries

```
/api/products/?stock=10
/api/products/?search=keyboard
/api/products/?ordering=-price
```

Advanced filtering

```
import django_filters


class ProductFilter(django_filters.FilterSet):
    min_price = django_filters.NumberFilter(
        field_name="price",
        lookup_expr="gte",
    )

    max_price = django_filters.NumberFilter(
        field_name="price",
        lookup_expr="lte",
    )

    class Meta:
        model = Product
        fields = ["min_price", "max_price"]
```

ViewSet-এ:

```
filterset_class = ProductFilter
```

Client-কে শুধু approved fields-এর ওপর filter এবং ordering করতে দাও।

---

21. Pagination

Pagination বড় result set-কে ছোট pages-এ ভাগ করে।

PageNumberPagination

```
from rest_framework.pagination import PageNumberPagination


class ProductPagination(PageNumberPagination):
```

```
    page_size = 10
    page_size_query_param = "page_size"
    max_page_size = 100
```

Apply:

```
class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    pagination_class = ProductPagination
```

Request:

```
/api/products/?page=2&page_size=20
```

Pagination types

PageNumberPagination

Page number-based pagination।

LimitOffsetPagination

```
from rest_framework.pagination import LimitOffsetPagination


class ProductPagination(LimitOffsetPagination):
    default_limit = 20
    max_limit = 100
```

Request:

```
/api/products/?limit=20&offset=40
```

CursorPagination

```
from rest_framework.pagination import CursorPagination


class ProductCursorPagination(CursorPagination):
    page_size = 20
    ordering = "-created_at"
```

Cursor pagination বড় বা পরিবর্তনশীল datasets-এর জন্য উপযোগী হতে পারে। Stable ordering গুরুত্বপূর্ণ।

---

22. Versioning

API versioning breaking changes ছাড়া API evolve করতে সাহায্য করে।

URL path versioning

Settings:

```
REST_FRAMEWORK = {
    "DEFAULT_VERSIONING_CLASS": (
        "rest_framework.versioning.URLPathVersioning"
    ),
    "DEFAULT_VERSION": "v1",
    "ALLOWED_VERSIONS": ["v1", "v2"],
}
```

URL:

```
path(
    "api/<str:version>/products/",
    ProductListCreateView.as_view(),
)
```

View:

```
version = request.version
```

Other versioning strategies

- `NamespaceVersioning`
- `AcceptHeaderVersioning`
- `QueryParameterVersioning`
- `HostNameVersioning`

Version policy ও backward compatibility documentation-এ স্পষ্ট রাখো।

---

23. Content Negotiation

Content negotiation response-এর representation নির্বাচন করে।

Client header:

```
Accept: application/json
```

Example:

```
from rest_framework.renderers import (
    JSONRenderer,
    BrowsableAPIRenderer,
)
```

```
class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

    renderer_classes = [
        JSONRenderer,
        BrowsableAPIRenderer,
    ]
```

`Content-Type` vs `Accept`

- `Content-Type`: Client request body কোন format-এ পাঠিয়েছে।
- `Accept`: Client কোন response format চায়।

Parser incoming body process করে; renderer outgoing representation তৈরি করে।

---

24. Metadata

Metadata endpoint-এর capabilities সম্পর্কে তথ্য দেয়। DRF-এ `OPTIONS` request-এর মাধ্যমে metadata পাওয়া যেতে পারে।

Settings:

```
REST_FRAMEWORK = {
    "DEFAULT_METADATA_CLASS": (
        "rest_framework.metadata.SimpleMetadata"
    ),
}
```

Example:

```
OPTIONS /api/products/
```

Metadata supported methods ও serializer-related field information-এর মতো তথ্য দিতে পারে। এটি পূর্ণাঙ্গ API documentation-এর বিকল্প নয়।

---

25. Schemas and OpenAPI

Schema API endpoints, operations, parameters, request bodies এবং response structures বর্ণনা করে।

drf-spectacular

Install:

```
pip install drf-spectacular
```

Settings:

```
INSTALLED_APPS += ["drf_spectacular"]

REST_FRAMEWORK = {
    "DEFAULT_SCHEMA_CLASS": (
        "drf_spectacular.openapi.AutoSchema"
    ),
}
```

URLs:

```
from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularSwaggerView,
)

urlpatterns += [
    path(
        "api/schema/",
        SpectacularAPIView.as_view(),
        name="schema",
    ),
    path(
        "api/docs/",
        SpectacularSwaggerView.as_view(
            url_name="schema"
        ),
        name="swagger-ui",
    ),
]
```

Benefits

- API documentation
- Swagger UI
- Frontend integration
- Client SDK generation
- Easier endpoint testing

Generated schema যাচাই করবে, বিশেষত custom actions ও custom request parameters-এর ক্ষেত্রে।

---

26. Format Suffixes

Format suffix URL-এর শেষে representation hint যোগ করে, যেমন `.json`।

```
from rest_framework.urlpatterns import format_suffix_patterns

urlpatterns = format_suffix_patterns([
    path(
        "products/",
        ProductListCreateView.as_view(),
        name="product-list",
    ),
])
```

Example:

```
/api/products.json
```

এটি optional feature। Router-based APIs-তে `DefaultRouter` ইতিমধ্যে format suffix support দিতে পারে।

---

27. Returning URLs

Django-এর `reverse()` named route থেকে URL তৈরি করে।

```
from django.urls import reverse

url = reverse(
    "product-detail",
    kwargs={"pk": product.pk},
)
```

DRF-এর `reverse()`:

```
from rest_framework.reverse import reverse

url = reverse(
    "product-detail",
    kwargs={"pk": product.pk},
    request=request,
)
```

Request context দিলে fully qualified URL তৈরি করা সম্ভব।

HyperlinkedModelSerializer

```
class ProductSerializer(
    serializers.HyperlinkedModelSerializer
):
    class Meta:
        model = Product
        fields = ["url", "id", "name", "price"]
```

Hyperlinked serializers ব্যবহার করার সময় সঠিক detail URL configuration এবং serializer request context নিশ্চিত করো।

---

28. Exceptions and Error Handling

DRF exceptions-কে HTTP error responses-এ রূপান্তর করে।

Common exceptions

Exception	Purpose
`ValidationError`	Invalid input
`NotAuthenticated`	Authentication required
`AuthenticationFailed`	Invalid credentials
`PermissionDenied`	Access denied
`NotFound`	Resource missing

`MethodNotAllowed`	Unsupported method
`NotAcceptable`	Unsupported response format
`UnsupportedMediaType`	Unsupported request content type
`Throttled`	Rate limit exceeded
`APIException`	Custom API errors

Example:

```
from rest_framework.exceptions import NotFound
from rest_framework.views import APIView


class ExampleView(APIView):

    def get(self, request):
        raise NotFound("Product not found.")
```

Custom exception handler

```
from rest_framework.views import exception_handler


def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
```

```
    if response is not None:
        response.data = {
            "error": response.data,
        }

    return response
```

Settings:

```
REST_FRAMEWORK = {
    "EXCEPTION_HANDLER": (
        "api.exceptions.custom_exception_handler"
    ),
}
```

এই handler DRF-এর handled exceptions-এর response format পরিবর্তন করে। Unexpected programming errors-এর জন্য logging ও monitoring রাখো।

---

29. HTTP Status Codes

Code	Meaning
200	OK
201	Created
202	Accepted
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found

405	Method Not Allowed
406	Not Acceptable
409	Conflict
415	Unsupported Media Type
429	Too Many Requests
500	Internal Server Error
503	Service Unavailable

Example:

```
from rest_framework import status
from rest_framework.response import Response


return Response(
    {"message": "Created"},
    status=status.HTTP_201_CREATED,
)
```

401 vs 403

- `401` : Authentication challenge বা failure।
- `403` : Access forbidden।

Unauthenticated request-এর exact status configured authentication scheme-এর ওপর নির্ভর করতে পারে।

---

30. Testing

DRF-এর testing tools:

- `APIClient`
- `APITestCase`
- `APIRequestFactory`
- `force_authenticate()`

Example tests

`api/tests.py`

```
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APITestCase

from .models import Product


class ProductAPITests(APITestCase):

    def setUp(self):
        Product.objects.create(
            name="Keyboard",
            price="1200.00",
            stock=10,
        )

    def test_product_list(self):
        response = self.client.get(
            reverse("product-list")
        )

        self.assertEqual(
            response.status_code,
            status.HTTP_200_OK,
        )

        self.assertEqual(len(response.data), 1)

    def test_create_product(self):
```

```
        response = self.client.post(
            reverse("product-list"),
            {
                "name": "Mouse",
                "price": "500.00",
                "stock": 20,
            },
            format="json",
        )

        self.assertEqual(
            response.status_code,
            status.HTTP_201_CREATED,
        )

        self.assertTrue(
            Product.objects.filter(name="Mouse").exists()
        )
```

Run tests:

```
python manage.py test
```

What should be tested?

- GET, POST, PUT, PATCH, DELETE
- Invalid input
- Authentication
- Permissions
- User-to-user data isolation
- Filtering
- Pagination
- Missing objects
- Unsupported methods
- Edge cases

`force_authenticate()` authentication flow bypass করে user simulate করতে পারে। Real token issuance এবং login flow test করতে আলাদা tests লিখবে।

---

31. Settings

DRF-এর global configuration `settings.py`-তে `REST_FRAMEWORK` dictionary-তে থাকে।

Example settings

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.SessionAuthentication",
        "rest_framework.authentication.BasicAuthentication",
    ],

    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticated",
    ],

    "DEFAULT_RENDERER_CLASSES": [
        "rest_framework.renderers.JSONRenderer",
        "rest_framework.renderers.BrowsableAPIRenderer",
    ],

    "DEFAULT_PARSER_CLASSES": [
        "rest_framework.parsers.JSONParser",
        "rest_framework.parsers.FormParser",
        "rest_framework.parsers.MultiPartParser",
    ],

    "DEFAULT_PAGINATION_CLASS": (
        "rest_framework.pagination.PageNumberPagination"
    ),

    "PAGE_SIZE": 20,

    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],

    "DEFAULT_THROTTLE_RATES": {
        "anon": "100/hour",
        "user": "1000/hour",
    },
}
```

Common settings

- `DEFAULT_AUTHENTICATION_CLASSES`

- `DEFAULT_PERMISSION_CLASSES`
- `DEFAULT_RENDERER_CLASSES`
- `DEFAULT_PARSER_CLASSES`
- `DEFAULT_THROTTLE_CLASSES`
- `DEFAULT_THROTTLE_RATES`
- `DEFAULT_PAGINATION_CLASS`
- `PAGE_SIZE`
- `DEFAULT_FILTER_BACKENDS`
- `DEFAULT_VERSIONING_CLASS`
- `DEFAULT_SCHEMA_CLASS`
- `EXCEPTION_HANDLER`
- `DEFAULT_METADATA_CLASS`

Global permissions `IsAuthenticated` করলে public endpoints-এ explicit public permission configure করতে হবে।

---

32. Complete Product CRUD API

এখানে Model, Serializer, ViewSet এবং Router একসঙ্গে ব্যবহার করা হয়েছে।

Step 1: Model

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
    stock = models.PositiveIntegerField(default=0)

    def __str__(self):
        return self.name
```

Step 2: Serializer

```
from rest_framework import serializers
from .models import Product


class ProductSerializer(serializers.ModelSerializer):
    class Meta:
```

```
        model = Product
        fields = ["id", "name", "price", "stock"]
        read_only_fields = ["id"]
```

Step 3: ViewSet

```
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAuthenticatedOrReadOnly

from .models import Product
from .serializers import ProductSerializer


class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Step 4: Router

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register("products", ProductViewSet, basename="product")

urlpatterns = router.urls
```

Step 5: Project URLs

```
from django.urls import path, include

urlpatterns = [
    path("api/", include("api.urls")),
]
```

Step 6: Database migrations

```
python manage.py makemigrations
python manage.py migrate
python manage.py runserver
```

Test with Postman

List

```
GET /api/products/
```

Create

```
POST /api/products/
Content-Type: application/json
```

```
{
  "name": "Keyboard",
  "price": "1200.00",
  "stock": 10
}
```

Partial update

```
PATCH /api/products/1/
Content-Type: application/json
```

```
{
  "stock": 25
}
```

Delete

```
DELETE /api/products/1/
```

Important: `IsAuthenticatedOrReadOnly` public reads এবং authenticated writes allow করে। Real project-এ object ownership, role restrictions এবং business-specific permissions যোগ করতে হবে।

---

33. Performance and Security Checklist

Performance

- `select_related()` ব্যবহার করো ForeignKey/OneToOne-এর জন্য।
- `prefetch_related()` ব্যবহার করো collection relations-এর জন্য।
- Large datasets paginate করো।
- N+1 query problem পরীক্ষা করো।
- দরকারি database indexes বিবেচনা করো।
- Expensive queries profile করো।
- Appropriate caching ব্যবহার করো।

Security

- Private endpoints-এ authentication enforce করো।
- Object-level authorization পরীক্ষা করো।
- Input validation ব্যবহার করো।
- Uploaded files validate করো।
- HTTPS ব্যবহার করো।
- Secrets source control-এ commit করো না।
- Rate limiting ও monitoring রাখো।
- Password বা tokens response-এ expose করো না।
- CORS ও CSRF configuration বুঝে করো।
- Database constraints ব্যবহার করো।
- Production error responses-এ stack traces প্রকাশ করো না।

---

34. Interview Questions and Answers

Q1. What is Django REST Framework?

Django REST Framework হলো Django-based toolkit যা REST APIs তৈরি করতে serialization, authentication, permissions, views, pagination ও testing support দেয়।

Q2. What is a serializer?

Serializer Python/model data-কে primitive representation-এ convert করে এবং incoming input validate করে।

Q3. Serializer vs ModelSerializer?

`Serializer`-এ fields ও persistence logic customভাবে define করা যায়। `ModelSerializer` Django model থেকে fields ও common validators তৈরি করতে পারে।

Q4. APIView vs GenericAPIView?

`APIView` HTTP methods ও DRF request lifecycle handle করে। `GenericAPIView` queryset, serializer ও object lookup-এর reusable behavior যোগ করে।

Q5. Generic views vs ViewSets?

Generic views সাধারণত specific endpoint behavior প্রকাশ করে। ViewSets resource-এর related actions এক class-এ রাখে এবং routers-এর সঙ্গে কাজ করে।

Q6. Authentication vs Permissions?

Authentication পরিচয় যাচাই করে। Permissions access control নির্ধারণ করে।

Q7. `request.data` vs `request.query_params`?

`request.data` parsed request body; `request.query_params` URL query string।

Q8. Parser vs Renderer?

Parser incoming data পড়ে। Renderer outgoing response representation তৈরি করে।

Q9. PUT vs PATCH?

PUT সাধারণত full resource replacement/update বোঝায়। PATCH partial update বোঝায়।

Q10. Why use `many=True`?

একাধিক model instances serialize করতে `many=True` ব্যবহার করা হয়।

Q11. What does `is_valid()` do?

Serializer-এর input validation চালায়। সফল হলে `validated_data`, ব্যর্থ হলে `errors` পাওয়া যায়।

Q12. Read-only vs Write-only fields?

Read-only field output-এ থাকে; write-only field input গ্রহণ করে কিন্তু output-এ দেখায় না।

Q13. Do object-level permissions filter list results?

না। List filtering-এর জন্য `get_queryset()` বা custom filtering দরকার।

Q14. `select_related()` vs `prefetch_related()`?

`select_related()` SQL joins ব্যবহার করে ForeignKey/OneToOne সম্পর্ক fetch করে।
`prefetch_related()` related collections-এর জন্য আলাদা queries ব্যবহার করতে পারে।

Q15. Is throttling sufficient for DDoS protection?

না। DRF throttling application-level rate control; infrastructure-level DDoS protection-এর বিকল্প নয়।

Q16. Why use pagination?

Response size এবং server workload নিয়ন্ত্রণ করে data ছোট chunks-এ দেয়।

Q17. Why API versioning?

Breaking changes থেকে existing clients-কে রক্ষা করে gradual migration করতে সাহায্য করে।

Q18. Why test permissions?

অনুমোদনহীন user যেন private data পড়তে বা পরিবর্তন করতে না পারে তা নিশ্চিত করতে।

Q19. What is content negotiation?

Client-এর `Accept` header ও configured renderers অনুযায়ী response representation নির্বাচন করা।

Q20. What makes a production-ready API?

Correct authorization, validation, testing, query optimization, secure settings, monitoring, documentation এবং consistent error handling।

---

35. Quick Revision Sheet

Requirement	**DRF component**
Parse request body	Parser
Validate input	Serializer
Serialize output	Serializer
Handle HTTP methods	APIView
Common CRUD endpoints	Generic views
Resource-based actions	ViewSet
Generate ViewSet URLs	Router
Identify user	Authentication

Control access	Permissions
Limit request rate	Throttling
Filter records	Filter backend
Split large results	Pagination
Create JSON response	Renderer
Describe API structure	OpenAPI schema
Test endpoints	APIClient / APITestCase

Recommended learning order

1. REST API fundamentals
2. Models and serializers
3. Function-based views and APIView
4. Generic views and mixins
5. ViewSets and routers
6. Authentication and permissions
7. Filtering and pagination
8. Testing
9. Query optimization and caching

10. Versioning and API documentation
11. Production deployment and monitoring

Official Documentation

- Main documentation: https://www.django-rest-framework.org/
- Quickstart: https://www.django-rest-framework.org/tutorial/quickstart/
- Requests: https://www.django-rest-framework.org/api-guide/requests/
- Responses: https://www.django-rest-framework.org/api-guide/responses/
- Serializers: https://www.django-rest-framework.org/api-guide/serializers/
- Generic views: https://www.django-rest-framework.org/api-guide/generic-views/
- ViewSets: https://www.django-rest-framework.org/api-guide/viewsets/
- Authentication: https://www.django-rest-framework.org/api-guide/authentication/
- Permissions: https://www.django-rest-framework.org/api-guide/permissions/
- Testing: https://www.django-rest-framework.org/api-guide/testing/
- Settings: https://www.django-rest-framework.org/api-guide/settings/

Final Advice

শুধু notes মুখস্থ না করে একটি বাস্তব Product Management API তৈরি করো। সেখানে CRUD, JWT authentication, owner-based permissions, filtering, pagination, automated tests এবং OpenAPI documentation যোগ করো।

প্রতিটি feature-এর success case এবং failure case test করলে Django REST Framework-এর practical knowledge ও backend interview preparation দুটোই শক্ত হবে।